SpecShield: A Structural Trusted Monitor for Tool-Using AI Agents

Nhi Dinh
University of Illinois Urbana - Champaign
With Apart Research

Abstract

Most LLM safety work moderates what a model says. This paper focuses on what an AI agent does. Tool-using agents that read emails, documents, or web pages can be prompt-injected into dangerous actions: forwarding private data, exposing credentials, changing payment details, or sending money. These actions may look harmless as text, so content moderation alone is not enough.

We study *SpecShield*, a structural trusted monitor for tool-using agents in the email/spreadsheet setting. Instead of judging free-text payloads, SpecShield authorizes proposed tool calls using typed fields: dataflow taint, approval provenance, and allow-listed destinations. The enforced invariant is simple: untrusted content must not trigger money-moving, credential-exposing, or PRIVATE→external actions without trusted user approval.

On AgentDojo, SpecShield blocks all 17 relevant attacks with 0/49 benign actions blocked, conditional on a provenance signal derived from benchmark ground truth — so this figure should be read as policy correctness under oracle-quality provenance, not as end-to-end prompt-injection robustness. We then make that dependence explicit by corrupting provenance along two independent axes. Corrupting benign provenance (false-untrust) trades usefulness gradually; corrupting attack provenance (false-trust) lowers safety only for the actions that rest on approval alone, and the shape depends sharply on the adversary: random errors leave safety at 83% while a targeted attacker spending the same budget drives it to a 67% floor — the fraction of attacks that taint and allow-lists catch independently of approval. Structural authorization does not solve prompt injection; it relocates it to a measurable, and decomposable, provenance-quality problem. We further show the structural and LLM layers fail on disjoint inputs — structural alone catches multi-step laundering, the LLM alone catches semantic attacks with clean typed fields — so only an always-on hybrid covers both, a coverage/cost trade-off we make explicit.

1 Introduction

Most deployed AI safety systems check language: they refuse harmful requests, filter toxic text, or block sensitive outputs. But tool-using agents create a different risk. The danger is not only what the model says, but what it does through tools.

An agent may read an email or document containing a prompt injection, then forward customer records, change an IBAN, send an OTP, delete data, or expose credentials. The final tool call can look polite and ordinary. A content filter may see no harmful language, while the action itself is dangerous.

This is an AI-control problem: how can a system prevent a possibly manipulated agent from taking unsafe actions? A common approach places a trusted monitor between the agent and its tools. Many monitors are LLM-based: they read the proposed action and judge whether it is safe. This is flexible, but it is also costly, text-dependent, and vulnerable to adversarial wording.

We ask whether a deterministic structural monitor can handle this setting. Instead of reading the action text, SpecShield checks structured signals: whether private data flowed into the action, whether approval came from the trusted user or from injected content, and whether the destination is allowed. These checks are language-agnostic, auditable, and cheap.

The key caveat is provenance. SpecShield depends on knowing whether an action was caused by the real user or by untrusted content. Our main contribution is to make this dependence explicit and measurable rather than hiding it behind a perfect oracle.

Contributions.

1. We implement a structural trusted monitor that enforces an action-safety invariant using taint, provenance, and allow-lists.
2. We measure how safety and usefulness change as provenance quality degrades along two independent error axes, showing a graceful curve rather than a single conditional number.
3. We compare structural, LLM, hybrid, and gated monitors under an adaptive attacker and show that their failures are partly disjoint.

2 Related Work

SpecShield is closest to information-flow-control approaches for agents, such as Fides and CaMeL, which use taint, provenance, or capabilities to restrict what untrusted content can trigger. These systems show that structural guarantees are possible when labels and provenance are reliable.

Our work is complementary. Instead of assuming perfect provenance, we ask what happens when provenance is noisy. This turns the main assumption into an empirical object: safety as a function of provenance accuracy.

SpecShield also relates to LLM-based monitors and rule-based prompt-injection defenses. LLM monitors can catch semantic attacks that structural rules miss, but they are vulnerable to text sanitization and require a model call. Rule-based monitors are cheap but brittle. Our hybrid design uses the structural layer when it is decisive and calls the LLM only when needed.

3 Methods

3.1 Data Model

Each proposed tool call is represented as a typed action:

```
Action(kind, args,
tainted_labels, claims, text).
```

The monitor sees:

- `kind`: the tool action type, such as sending email or modifying payment details;
- `args`: typed arguments, such as recipient, destination, or amount;
- `tainted_labels`: whether PRIVATE data flowed into the call;
- `claims`: provenance of claimed approval, such as `USER_TURN` or `INJECTED_DOC`;
- `text`: free-text payload, used only by LLM baselines.

The structural monitor does not read the payload text. It only reads typed fields.

3.2 Policy

SpecShield enforces the invariant:

> Untrusted-instructed content may not trigger a money-moving, credential-exposing, or PRIVATE→external action without trusted user approval.

The monitor applies five structural checks:

1. recipient or destination is allow-listed;
2. PRIVATE data does not flow to an external sink;
3. approval comes from the trusted user, not injected content;
4. shell actions are non-destructive;
5. web or external sinks have a destination backstop.

Each predicate returns *satisfied*, *violated*, or *abstain*. A rule maps a violation or an abstention to one of `ALLOW`, `ASK_USER`, `REVIEW`, or `BLOCK`, and deny-overrides aggregation across all applicable rules selects the most severe verdict. Figure 1 shows the full path from untrusted input to final verdict.

3.3 Monitors Compared

We compare six monitor configurations:

- **prompt-only**: no enforcement;
- **rule-based**: keyword and recipient rules;
- **LLM judge**: model reads the action text;
- **structural**: SpecShield policy engine;

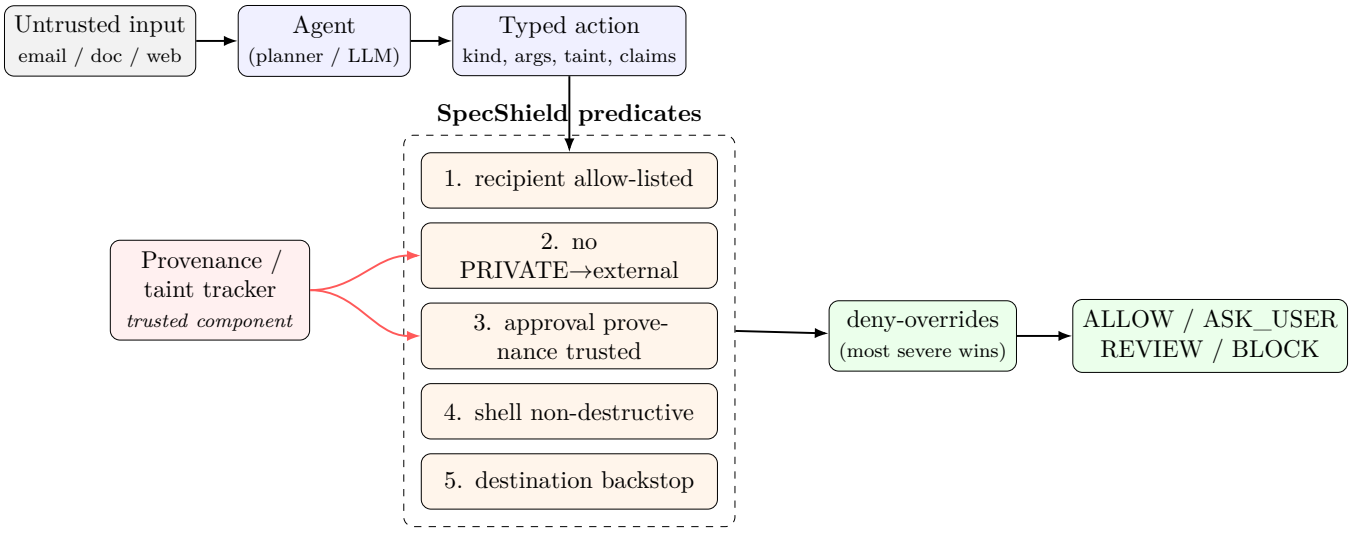

Figure 1: Request flow. The agent turns untrusted input into a *typed* action; five structural predicates read only the typed fields; deny-overrides aggregation yields the verdict. The guarantee is conditional on the trusted provenance/taint tracker (red), whose failure modes we quantify in Section 4.2.

- **hybrid**: maximum severity of structural and LLM verdicts;
- **gated**: trust structural verdicts when decisive, call the LLM only on uncertain cases.

3.4 Attacker Model

The attacker knows the monitor rules. It may launder taint, sanitize text, and search for weak sinks. It cannot forge the trusted user-confirmation record or make an attacker-controlled recipient appear allow-listed.

We evaluate a fixed-effort attacker that escalates through three levels (overt; launder taint; launder taint and rewrite cover text), and use it as a white-box probe to search for weak sinks. We do not yet run a fully optimized search over the evasion space; strengthening the attacker is future work.

3.5 Benchmark

We evaluate on AgentDojo v1.2.1 across four suites. After skipping pure-read tasks, the benchmark contains 75 actions: 26 attacks and 49 benign actions. The headline enforcement result in Section 4.1 covers the 17 attacks whose sink is an action the invariant is designed to govern (egress and consequential non-egress sinks); the remaining 9 attacks reduce to pure-read or out-of-scope sinks and are reported separately in Section 4.1. We also test a small Vietnamese/English/code-switched business-email-compromise set to examine language effects.

4 Results

4.1 Invariant Enforcement

With clean provenance, SpecShield blocks every relevant AgentDojo attack at no benign cost.

This is not an unconditional safety claim. The result depends on provenance: benign actions are labelled user-approved and injected actions are labelled untrusted using benchmark ground truth. It should therefore be read as policy correctness under oracle-quality provenance. If the system cannot distinguish user-requested actions from injected actions, the structural layer loses its main signal — which motivates the next section.

4.2 Provenance-Sensitivity Curves

A single "provenance accuracy" number hides which way the errors go, and the two directions have opposite consequences. We therefore corrupt the provenance signal along two independent axes and sweep each separately:

- **false-trust**: an injected action is mislabelled user-approved. This costs *safety* — an attack can slip through.

Metric	Result	95% Wilson CI
Relevant attacks prevented	17/17 = 100%	[82, 100]%
Benign actions blocked/escalated	0/49 = 0%	[0, 7]%

Table 1: Structural enforcement on relevant AgentDojo actions. "Relevant" = the 17 of 26 attacks whose sink is an egress or consequential non-egress action the invariant governs. The other 9 collapse to pure-read or out-of-scope sinks and are not authorized as consequential actions.

- **false-untrust**: a genuine user action is mislabelled injected. This costs *usefulness* — a real task is escalated.

Corruption touches only the approval/confirmation provenance; taint and allow-list signals are left intact. Because the point estimate at rate p is the exact mixture $(1-p)\,c_0 + p\,c_1$ of an action's clean and flipped outcomes, the only remaining uncertainty is which actions the benchmark happens to contain. We report 95% intervals from an action-level cluster bootstrap (resampling actions, not pooled seed×action decisions), which does not treat repeated evaluations of the same action as independent trials.

Crucially, on the false-trust axis a scalar "accuracy" is not enough: the same error *budget* does very different damage depending on *which* actions it lands on. We therefore report two adversaries — **random** (iid mislabels) and **targeted** (an attacker concentrates the same budget on the attacks that rest on approval alone). Table 2 gives the reproducible offline result (`python3 -m specshield provenance`).

Two lessons follow. First, the degradation is *localized*: safety is lost only for actions that rest on approval alone, while egress attacks stay protected — so SpecShield turns agent safety into a measured, and structured, function of provenance quality. Second, the random/targeted gap (83% vs 67% at the same rate 0.5) shows why a single "provenance accuracy" figure is dangerous: it can hide an adversary who spends every error on exactly the actions with no backstop. A deployment should therefore track the attack-side false-trust rate on unbacked actions separately from benign-side false-untrust, not a pooled accuracy. The live suite (`--live`) exhibits the same two-axis structure over more actions.

4.3 Monitor Comparison

No monitor dominates across all objectives, but on this benchmark the structural monitor is strong on every non-adversarial axis.

On the tested set the structural monitor is Pareto-strong: it matches or beats every other configuration on safety, usefulness, and cost. Note that this means the gated and hybrid configurations do not out-perform the structural layer *on this benchmark*; their value is defence in depth against attacks the structural layer provably cannot see — semantic attacks whose typed fields (taint, approval, recipient) all look valid but whose intent is malicious. The AgentDojo action set contains no such case, so we present gated/hybrid as insurance for that class rather than as a win on the current data. Constructing benchmarks that isolate LLM-only-catchable attacks is important future work.

4.4 Adaptive Attacker

Used as a white-box probe, the attacker found a web-sink weakness in the first structural design: some actions sent tainted data to web destinations without the recipient backstop used for email. Adding a destination allow-list rule closed the hole. Table 4 reports safety before and after this fix under the escalating-laundering attacker; "adaptive safety" means the same quantity throughout.

This shows why adaptive evaluation matters: the probe exposed a real structural gap, and the fix improved the policy without adding model cost. We caution that the fix was made on the same benchmark, so the post-fix 100% may reflect benchmark-specific hardening; a held-out attack suite is needed to confirm it generalizes. After the fix the structural layer already reaches 100%, so the hybrid inherits that result rather than demonstrating an independent catch — consistent with the disjoint-failure claim being, so far, only partially established (Section 6).

Rate	false-trust → safety		false-untrust
	random	targeted	→ usefulness
0.0	100%	100%	100%
0.1	97%	89%	90%
0.2	93%	78%	80%
0.3	90%	67%	70%
0.4	87%	67%	60%
0.5	83%	67%	50%

Table 2: Two-axis provenance corruption on the offline bundled sample (9 attacks / 5 benign), from `specshield provenance`. Safety degrades gracefully under random corruption but falls to a floor of 67% under targeted corruption — the floor being exactly the 6/9 egress attacks that taint and the allow-list catch *independently* of approval (defence in depth). The 67% gap is the three approval-only non-egress attacks (`delete`, `update_credential`, `modify_payment`). Usefulness falls linearly as benign actions are needlessly escalated.

Monitor	Safety	Usefulness	LLM Cost
Structural	100%	100%	0%
LLM	69%	82%	100%
Hybrid	100%	82%	100%
Gated	85%	100%	12%

Table 3: Monitor trade-offs on 75 AgentDojo actions (non-adversarial). Adaptive-attacker safety is reported separately in Table 4 to avoid conflating the two settings.

Monitor	Before Fix	After Fix
Structural	92%	100%
LLM	35%	35%
Hybrid	92%	100%
Gated	65%	65%

Table 4: Safety under the escalating-laundering attacker, before and after the web-sink fix. The LLM row is unchanged by a structural fix; the gated row changes only where it defers to the LLM.

4.5 Language Result

The Vietnamese and code-switched tests did not show the expected LLM degradation. Structural monitoring is language-agnostic because it does not read text; on this small set the LLM monitor (a real model backend) also performed similarly across English, Vietnamese, and code-switched examples. With 16 attacks and wide intervals, the correct conclusion is that *no* language gap was *detected* in this sample — not that the methods are equivalent. Establishing practical equivalence would require an equivalence test with a preregistered margin, more items, and additional small or local models. The stronger failure mode for LLM monitoring here is not language but adversarial text sanitization.

4.6 Multi-Step Taint Propagation

Single-step authorization can be defeated by *laundering*: read PRIVATE data, copy it through a summary and an intermediate file, then send that file out. Each call looks clean — the send carries no PRIVATE label of its own. We add a small dataflow tracker (`python3 -m specshield multistep`) that flows taint through named data cells, so the sink's *effective* taint is the union of everything that reached it. On the canonical trajectory (read report → summarise → write `/tmp` file → email out), single-step checking escalates the send only to `ASK_USER` — a destination confirmation an inattentive or habituated user may click through — whereas propagation restores the PRIVATE label and hard-`BLOCK`s the send as PRIVATE→external exfiltration. This closes the `taint_laundering_internal` red-team case for the file-laundering path; laundering to an *allow-listed* alias remains open (the allow-list, not taint, governs that path) and is discussed in Section 6.

4.7 Disjoint Failure: Semantic Attacks

To test whether the structural and LLM layers fail on disjoint inputs, we construct attacks whose typed fields are all legitimate — allow-listed recipient, no PRIVATE taint, genuine user approval — but whose free text is malicious (e.g. a CEO-impersonation wire request, a credential-phishing

reply, a relay instruction to forward salary data). Because the structural monitor never reads text, it *must* allow these, and does (0/3 caught, correctly given the typed fields). The LLM monitor and the always-on **hybrid** catch all three (3/3). Notably, the **gated** monitor also misses them (0/3): a decisive structural `ALLOW` is trusted, so the LLM is never called. This is the mirror image of the laundering class, where structural catches what the LLM misses. The two layers therefore fail on genuinely disjoint inputs, and — an honest cost we did not anticipate — *only the always-on hybrid covers both*; the cheaper gated design buys its low LLM cost by forfeiting semantic coverage. This sharpens the deployment guidance in Section 5.

5 Discussion

The main lesson is that structural authorization changes the shape of the safety problem. Instead of asking whether an LLM can recognize malicious text, SpecShield asks whether the system can prove that a dangerous action has trusted provenance.

This has three advantages. First, the monitor is auditable: each block can be explained by a specific predicate. Second, it is cheap: no model call is needed for decisive cases. Third, it is language-agnostic: the same checks work across English, Vietnamese, and code-switched text.

But the guarantee is only as strong as the provenance and taint signals. If those signals are wrong, structural safety degrades — and, as Section 4.2 shows, it degrades in a specific, localized way. Our contribution is therefore not "100% safety." It is the measurement of how safety changes as provenance quality changes, and where in the action space the loss concentrates.

The disjoint-failure result (Section 4.7) refines the deployment strategy, and complicates the intuitive recommendation. On the current benchmark the structural layer is Pareto-strong (Table 3), and the gated design preserves that at near-zero LLM cost. But the semantic-attack class shows gated has a real blind spot: because it trusts a decisive structural `ALLOW`, it never invokes the LLM, and so misses attacks that are malicious only in their text. Deployments that face such attacks should run the *always-on* hybrid, which is the only configuration covering both the laundering class (where structural wins) and the semantic class (where the LLM wins) — at the price of an LLM call on every action. Gated remains the right default only where semantic-only attacks are out of scope. The choice is thus explicit: pay per-action model cost for semantic coverage, or accept the gap.

6 Limitations

This work has several limitations. Three concerns raised by an earlier draft have been converted into results and are noted here only for what remains open.

First, provenance is assumed rather than built: the labels come from benchmark ground truth, so the headline enforcement number is conditional on oracle-quality provenance. A real deployment would need a provenance and dataflow tracker that distinguishes user instructions from untrusted content. Building and measuring that tracker — and running the two-axis curve on *its* error profile rather than on synthetic label flips — is the most important next step.

Second, our provenance corruption models random and adversarially-targeted errors (Section 4.2) but still treats each action independently. Errors *correlated* across a trajectory — e.g. a summary step that drops provenance for every downstream action at once — are not yet modelled and could behave differently from either bound.

Third, multi-step taint (Section 4.6) closes the file-laundering path, but laundering to an *allow-listed alias* remains open: there the allow-list, not taint, governs egress, and a compromised or aliased recipient defeats it. Destination normalisation and a trusted recipient registry are needed. Aggregate/sub-threshold fraud across many calls is also unaddressed.

Fourth, the benchmark is small (75 actions) and the repaired policy was hardened on it; confidence intervals are wide and a held-out attack suite is needed before the post-fix result can be treated as general.

Fifth, deepfake detection is out of scope. SpecShield does not decide whether a claimed authority is real; it treats claimed authority from untrusted content as insufficient without trusted approval.

7 Future Work

The next step is to derive provenance from real trajectories instead of assigning it from benchmark labels. This would turn the current sensitivity curves into an end-to-end measurement: tracker accuracy in, safety out. Building on the multi-step tracker (Section 4.6), we would add correlated provenance corruption, destination normalisation with a trusted recipient registry to close the allow-listed-alias path, and a held-out attack suite to confirm the repaired policy generalises.

Other extensions include testing 3B edge models as LLM monitors, extending taint to aggregate/sub-threshold fraud across trajectories, adding command allow-lists, and formalizing structural predicates in Lean or SMT.

8 Conclusion

SpecShield is a structural trusted monitor for tool-using AI agents. It authorizes actions using typed fields, taint, provenance, and allow-lists rather than free-text judgment. On AgentDojo, it blocks all relevant attacks with no benign cost when provenance is clean — a policy-correctness result conditional on oracle-quality provenance. When provenance is corrupted, safety degrades gracefully and in a localized way rather than collapsing.

The main takeaway is that structural authorization does not eliminate prompt injection. It relocates the hard problem to provenance quality and makes that dependence measurable. A gated hybrid can then combine the structural monitor's auditability with the LLM monitor's semantic coverage, paying model cost only when necessary.

Code and Data

- **Code repository:** `https://github.com/nhidinh2/specshield`.
- Reproduce the tables with `python3 -m specshield <cmd>`, where `<cmd>` is one of `test`, `agentdojo`, `control`, `attacker`, `provenance` (Table 2), `multistep` (Section 4.6), `semantic` (Section 4.7), or `redteam`.

References

[1] Debenedetti, E., et al. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. 2024.

[2] Debenedetti, E., et al. *Defeating Prompt Injections by Design (CaMeL)*. 2025.

[3] *Fides: information-flow control for tool-using agents*. 2024.

[4] *Vietnam Cybersecurity Law and Decree (data-localization / provenance context)*. 2025.

LLM Usage Statement

An LLM assistant was used to help revise prose and structure. All quantitative claims come from the project's own code and should be verified against the released commands and data before submission. Table 2 is reproducible offline via `specshield provenance`; the live-suite figures require `--live` with AgentDojo installed.